

Further Applications of Hashing (With Fewer Tangents...)

Design and Analysis of Algorithms

Brian C. Dean

**School of Computing
Clemson University**



Pseudo-Random Number Generation

- True random numbers are impossible to generate on a deterministic computer.
- Hash functions give us a simple way to generate pseudo-random numbers, though, since they are designed to look as “random” as possible.
- Just evaluate $h(0)$, $h(1)$, $h(2)$, ... to get a nice stream of pseudo-random numbers.

Estimating the # of Distinct Elements in a Large Data Stream

- Your program monitors a set of N integers streaming off a sensor very quickly.
- How can you determine the number of distinct integers that go by?

**Store them all in a hash table, skipping duplicates.
 $O(N)$ time.**

- Now what if you don't have enough memory to store the hash table (say N is very large)?

Estimating the # of Distinct Elements in a Large Data Stream

- If the D distinct numbers streaming by were random, then we would expect:
 - ~ $D/2$ of them to be even (ending in 0 in binary).
 - ~ $D/4$ of them to be multiples of 4 (ending in 00).
 - ~ $D/8$ of them to be multiples of 8 (ending in 000).
 - etc.
- For example, if we see numbers ending in 00 but none ending in 000, we might guess $D = 4$.

Estimating the # of Distinct Elements in a Large Data Stream

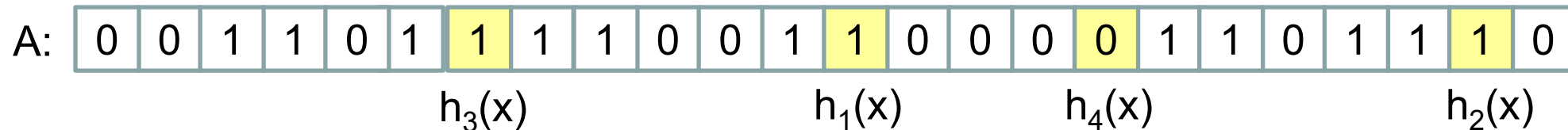
- The numbers in our input are not randomly distributed, so the preceding technique would not necessarily work.
- However, we can make our numbers look random by looking at their hashes instead.
 - And this doesn't appreciably change the number of distinct elements!

Another Streaming Application: Estimating Element Frequencies

- Suppose N integers stream by.
- We would like to estimate the frequency f_v (number of occurrences) of a value v .
- This is easy to do if we have a lot of memory, but what if memory is limited?
- Using a “count min sketch”, we can get an estimate in the range $[f_v, f_v + \epsilon N]$ with high probability using only $O(\epsilon^{-1} \log N)$ space!

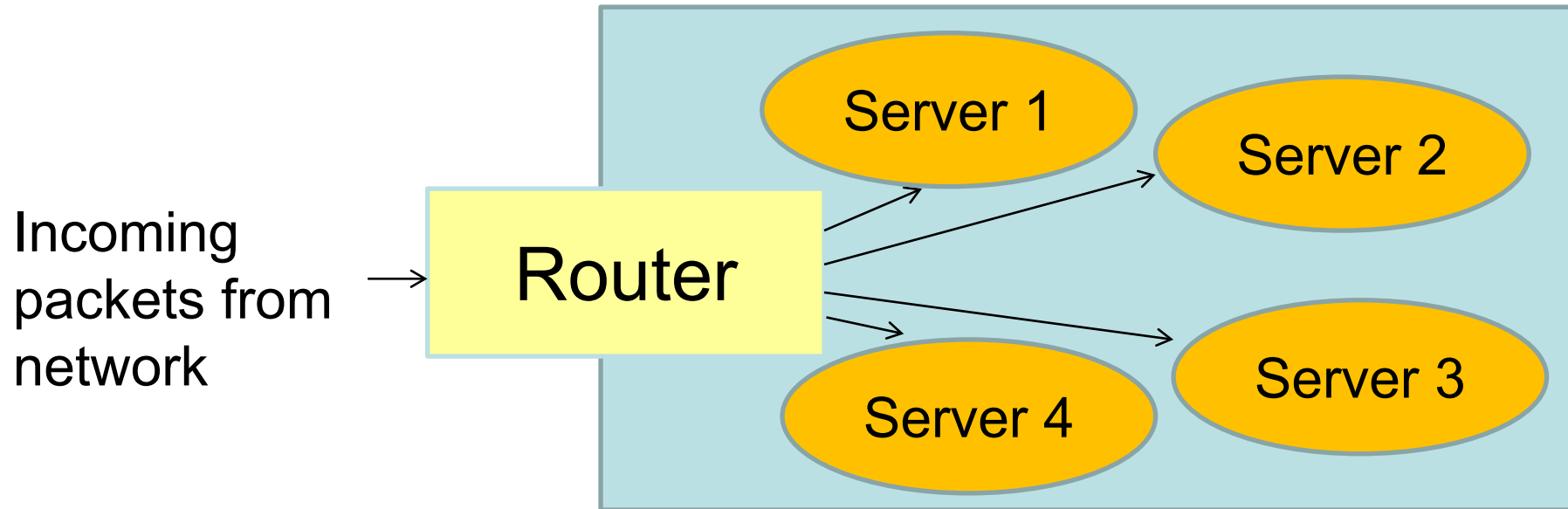
Low-Memory Approximate Membership Testing: Bloom Filters

- Insert(x): set $A[h_1(x)] \dots A[h_k(x)]$ all to 1.
- Find(x): examine $A[h_1(x)] \dots A[h_k(x)]$
 - If any of them are zero, x definitely not in the structure.
- Tradeoff between space and find accuracy; only needs a small number of bits times n for reasonable accuracy.
- No dependence on complexity of objects being stored.



Load Balancing

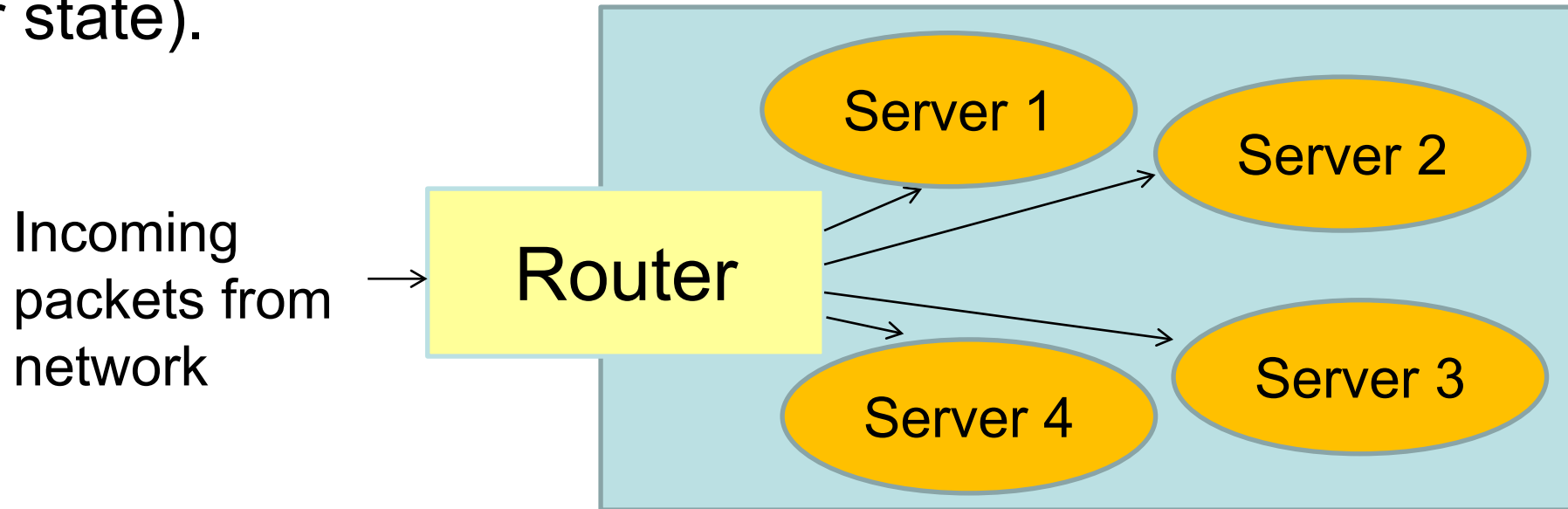
- Large websites often consist of multiple servers sitting behind a router.



- We' like to balance the load assigned to these servers...

Load Balancing

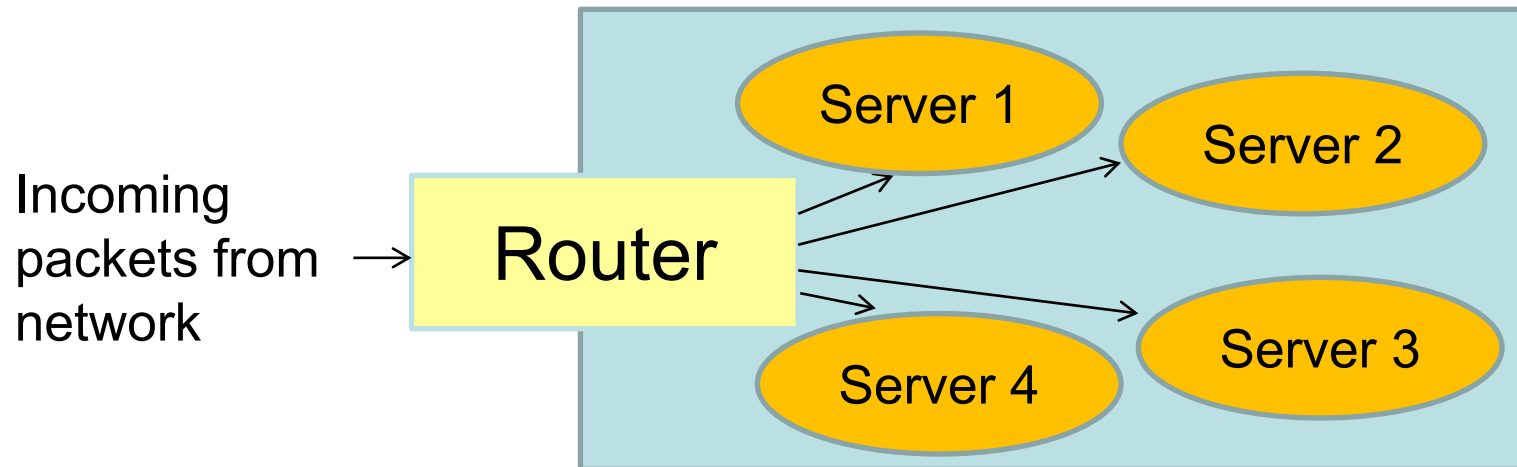
- Random assignment would balance the load.
- However, it doesn't consistently assign the same incoming source IP to the same server (useful if servers maintain “shopping carts” or other state).



- Good alternative solution: map packet with source IP address A to server $h(A)$.

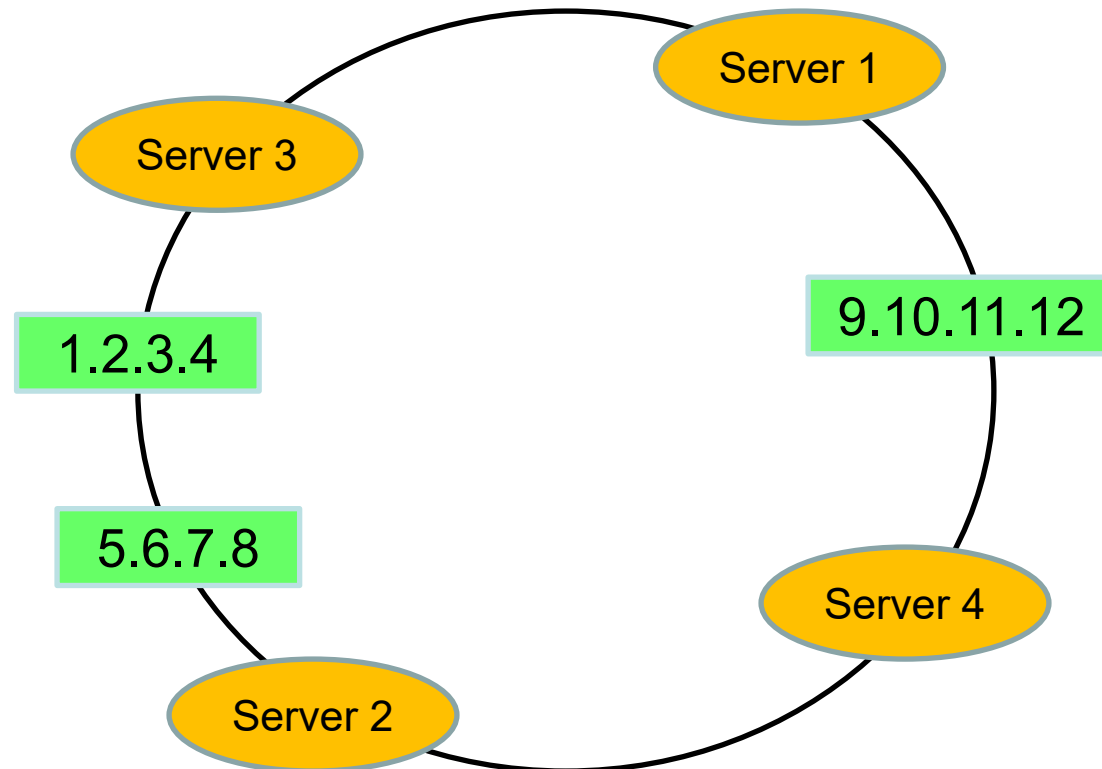
Consistent Hashing: Motivation

- Ok, so we are mapping packets with source IP address A to server $h(A)$.
- What if servers fail unexpectedly, or are removed / added to the website.
- How can we update our assignments now?



Consistent Hashing: One Approach

- Hash source IPs AND servers to a circle.
 - I.e., instead of mapping keys $\rightarrow$ table cells, map both the keys and cells to a common space!

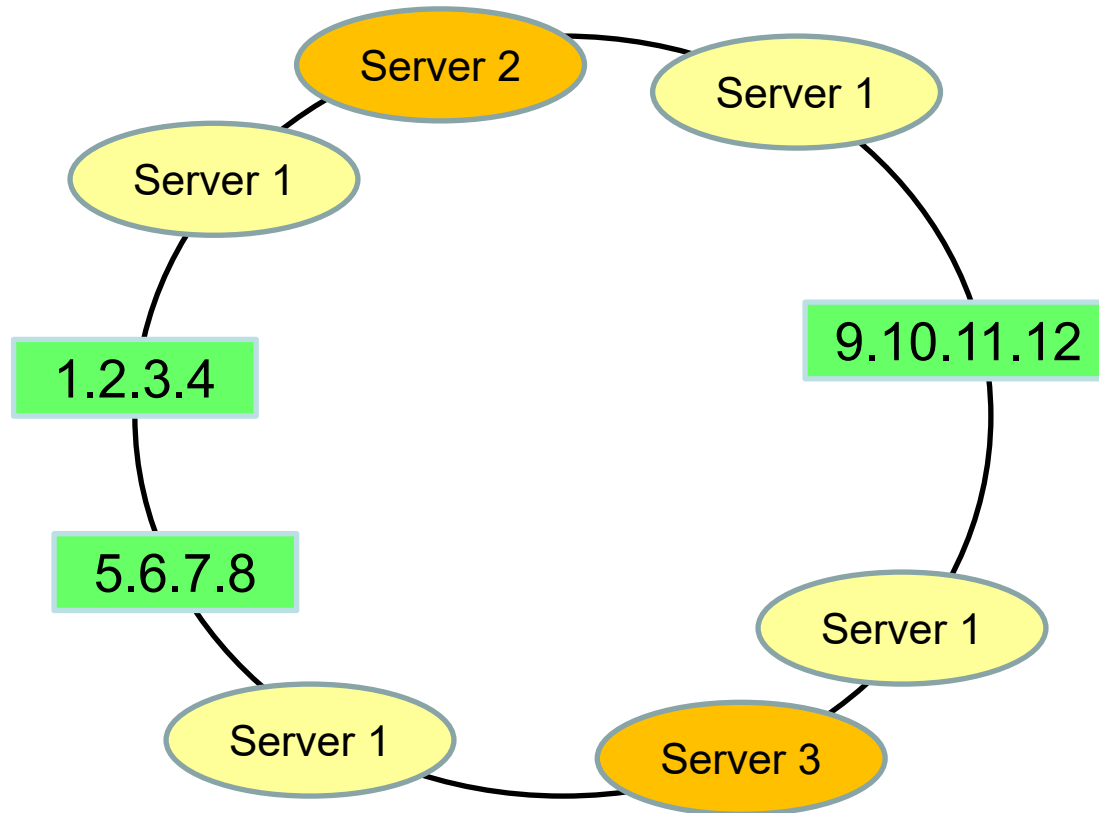


Each IP address is now assigned to the next server in clockwise rotation from it on the circle.

Does this fix the problem of servers failing?

Consistent Hashing: One Approach

- Why not map several instances of each server to the circle...



Now load from a failing server is spread more uniformly across the other servers.

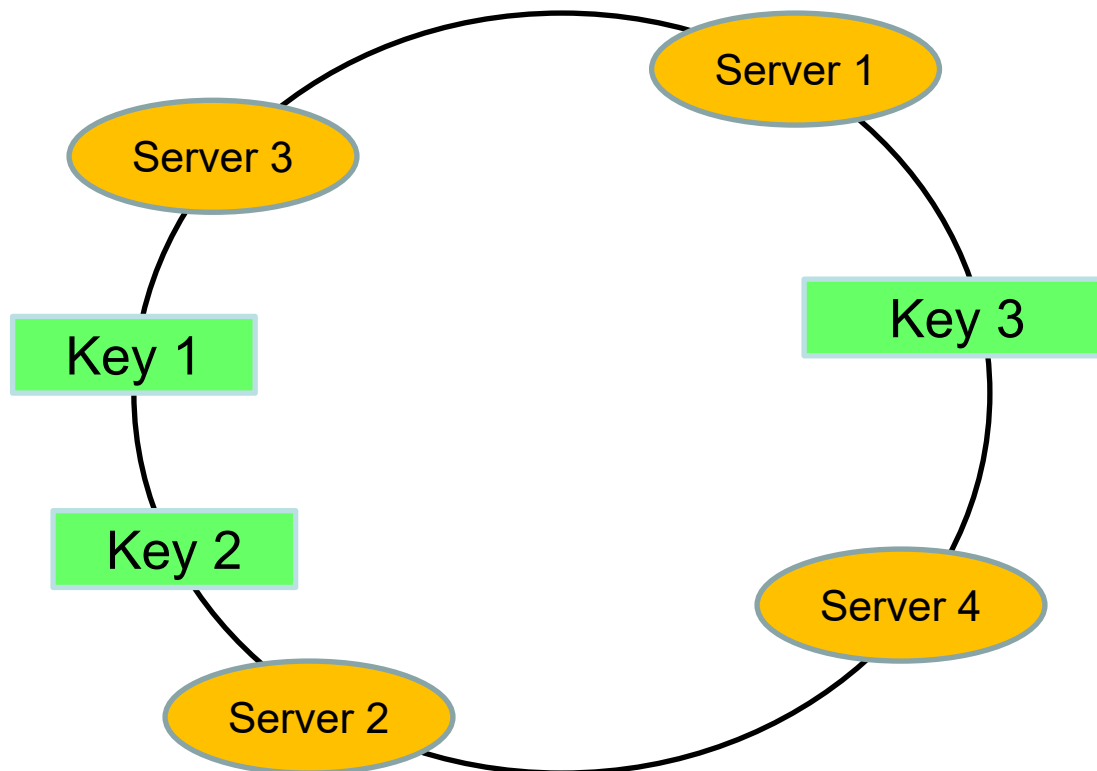
So we have consistency in our assignments as well as some fault tolerance.

Distributed Hash Tables (DHTs)

- What if we want to store a huge set of (key, value) pairs using a distributed network so that we can still perform *insert*, *remove*, and *find*?
- Ideally a decentralized solution, which does not depend on a small set of “root” servers that effectively know the address of the server on which every object is stored.

The “Chord” DHT

- Hash object keys and servers to a circle.
- Objects stored on the next clockwise server.

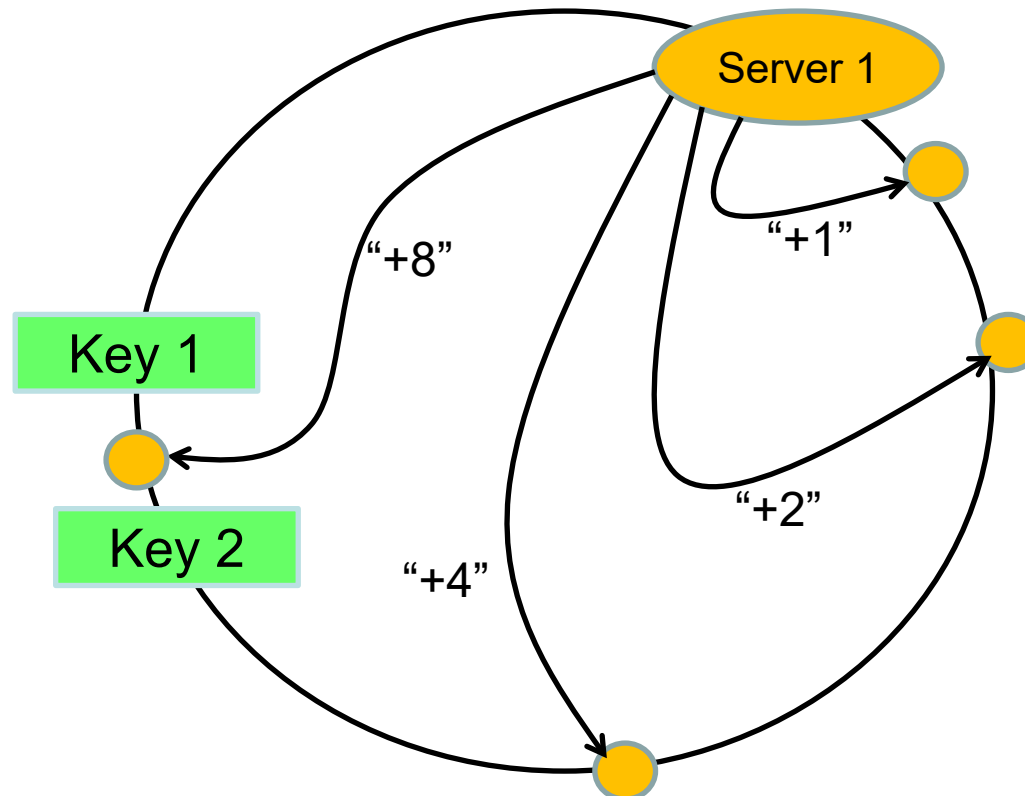


Can store multiple copies of each server and each key for fault tolerance purposes.

We instead must be able to initiate an operation (insert, remove, find) by contacting an arbitrary server...

The “Chord” DHT

- Use longer-range links, like in a skip list!
- Each node points to a collection of servers around the circle...



If each server maintains $O(\log N)$ links to successors at distances 1, 2, 4, 8, etc., then we can reach our final destination with only $O(\log N)$ hops!